

Optimizing F1 Pit Stop Strategy Under Safety Car Uncertainty Using Simulation and Genetic Algorithms

Yash Rahrkar

Washington University in St. Louis

St. Louis

r.yash@wustl.edu

github.com/Yashrah/F1optimization/

Abstract—Formula 1 race strategy depends on optimizing tire use and pit stop timing, decisions much influenced by random events like Safety Car (SC) deployments. This project creates a framework to find ideal dynamic pit stop plans for a single car confronting SC randomness. We define a strategy using two parameters: K , the earliest lap to pit under an SC, and M , the mandatory latest pit lap. The objective is to minimize the expected total race time, evaluated via Monte Carlo simulations incorporating data-driven tire degradation models and track-specific SC probabilities. Attempts to derive data-driven tire models from real F1 session data (Australian GP, Monaco GP, Japanese GP, Bahrain GP) were hindered by insufficient clean laps within targeted stints after necessary filtering. Due to the simulation-based, non-convex nature of the expected time function over integer variables (K, M) , a Genetic Algorithm (GA) implemented using MATLAB's Global Optimization Toolbox is employed. Results demonstrate that the optimal (K, M) strategy varies significantly based on track characteristics, particularly SC likelihood and race length, highlighting the necessity of adaptive approaches to optimization in dynamic environments. Visualizations illustrate the methodology and findings.

Index Terms—Optimization, Formula 1, Race Strategy, Genetic Algorithm, Monte Carlo Simulation, Stochastic Optimization, Pit Stop Strategy, Tire Degradation, Simulation Modeling.

I. INTRODUCTION

A. Background: The Strategic Complexity of F1

Formula 1 racing is a great feat of engineering and human performance, where making small decisions under pressure is very important. Races are won and lost not only on driver skill and car performance but also through astute strategy calls made from the pit that can impact the broader race. K Important choices concern timing of pit stops [?] and tire management (selection among Soft, Medium, and Hard compounds with different performance and durability). A standard pit stop costs significant time (typically 20s to 25s), which means teams must carefully plan to balance track position loss against the benefit of fresh tires.

Unpredictable events often throw off this delicate equilibrium; most famously, this is the deployment of the Safety Car (SC) or Virtual Safety Car (VSC) following track-based incidents. An SC period neutralizes the race, grouping the field and greatly lowers the time penalty connected with a

pit stop—usually by more than half. Should a team follow its pre-race strategy or stray to take advantage of a "cheap" SC pit stop, even if it is earlier or later than originally intended, this creates a strategic conundrum. Making the wrong call might cost several spots and maybe the race. Teams use sophisticated simulation tools to examine these situations, but the natural uncertainty makes optimization difficult. [4].

B. Problem Statement and Research Question

Traditional deterministic race strategy planning (e.g., "pit on lap 25") fails to account for the potential advantages offered by stochastic SC events. This project addresses this gap by formulating and solving an optimization problem for a dynamic pit stop strategy under SC uncertainty.

My guiding question is: *How can data from specific F1 race weekends be used to model tire performance and stochastic events (Safety Cars) to determine an optimal dynamic first pit stop strategy that minimizes expected total race time?*

We define a dynamic strategy using two integer parameters:

- K : The **SC Pit Threshold Lap**. The earliest lap at which the team will choose to pit if an SC is deployed.
- M : The **Mandatory Pit Lap**. The latest lap the team will execute the first pit stop if no SC has occurred between lap K and $M - 1$.

The objective function is the *expected* total race time, $E[\text{Time}(K, M)]$, calculated via simulation, and the goal is to find the pair (K^*, M^*) that minimizes this value, subject to physical and logical constraints ($1 \leq K < M \leq \text{NumLaps}$).

C. Optimization

This project directly engages with concepts central to optimization theory, as explored in ESE 415 [2]:

- **Problem Formulation:** Defining objective functions ($E[\text{Time}(K, M)]$), decision variables (K, M) , and constraints ($1 \leq K < M \leq \text{NumLaps}$).
- **Optimization Under Uncertainty:** Using expected value minimization for a stochastic system.
- **Simulation-Based Optimization:** Addressing objectives where analytical forms and gradients are unavailable.

- **Non-Convex Optimization:** Recognizing the likely non-convex nature of the objective function over integer variables, justifying the use of a global heuristic (GA) over local, gradient-based methods
- **Data-Driven Modeling Challenges:** Demonstrating the practical difficulties in obtaining suitable parameters from real-world data, a common issue contrasting with idealized problems often having known functions
- **Constrained Optimization:** Implementing bounds and inequality constraints on decision variables.

II. METHODS

Using Matlab, I developed a simulation and optimization framework

A. Simulation Environment (*main_optimize_strategy.m*)

The main script orchestrates the process:

- 1) Configures analysis parameters (tracks, data paths).
- 2) Loops through selected tracks.
- 3) Calls functions to load track-specific parameters (*define_race_parameters.m*) and attempts to fit tire models (*define_data_driven_tire_models.m*), falling back to placeholders (*define_placeholder_tire_models.m*) if fitting fails.
- 4) Calls the optimization function (*find_optimal_strategy.m*) to find the best (K, M) for the current track using the loaded/placeholder tire models.
- 5) Stores results and generates visualizations for each track.
- 6) Generates final multi-track comparison plots.

B. Data-Driven Tire Modeling Attempt

(*fit_tire_model_from_csv.m*) Realistic tire modeling seemed like an important feature to add, but proved challenging with the available data structure. The implemented process aimed to:

- 1) **Load Data:** Use *readtable* for a specific session CSV (sourced via FastF1 [1]).
- 2) **Filter Data:** Sequentially filter for a target driver, compound, and stint number. Apply crucial filters to retain only valid green-flag racing laps: remove inaccurate laps (*IsAccurate==false*), pit laps (*PitInTime/PitOutTime*), non-green flag laps (*TrackStatus = 1*), and time outliers.
- 3) **Fit Model:** If sufficient laps remain ($i \geq 3$), fit a quadratic model ($LapTime = p_1 \cdot age^2 + p_2 \cdot age + p_3$) using robust regression or *polyfit*.
- 4) **Extract Parameters:** Derive *base_lap_time* and *degradation_func*.
- 5) **Visualize Attempt:** Plot filtered data and the fit if successful (e.g., Fig. 1).

Challenge Encountered: The filtering process consistently reduced the number of usable laps for the targeted single stints (e.g., Stint 1 for VER/Medium) to zero or near-zero.

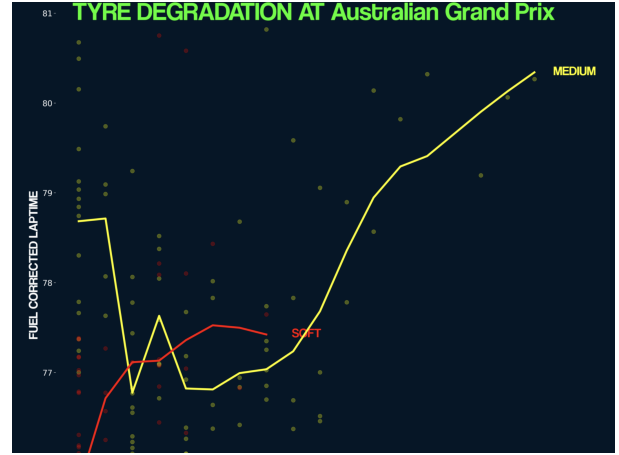


Fig. 1: Tire degradation data from Australian 2025 Grand Prix.

This prevented the successful fitting of data-driven models for the primary analysis runs. Consequently, the optimization relied on the pre-defined **placeholder tire models** (defined in *define_placeholder_tire_models.m*) which use example base times and degradation functions (e.g., linear or simple power laws). Fig. 1 shows that the F1 data site we scraped for data did have some type of clean tire degradation data, but I don't think it was publically available or redacted from their lap data.

C. Stochastic Simulation (*evaluate_strategy.m*)

This function estimates $E[\text{Time}(K, M)]$ via Monte Carlo, using the determined tire models (placeholders in this case):

- 1) Takes (K, M) , *race_params*, *tire_models* as input.
- 2) Runs *num_simulations_for_expected_time* (e.g., 1000) independent race simulations.
- 3) Simulates lap-by-lap progression, calculating lap times based on the (placeholder) tire model's *base_lap_time* and *degradation_func(age)*. Incorporates random SC deployment based on track-specific probability, applies SC lap time penalties, and executes the (K, M) pit stop logic, adding relevant pit time loss.
- 4) Returns the mean total time across all simulations. Optionally returns the array of all simulation times.

D. Optimization via Genetic Algorithm

(*find_optimal_strategy.m*) MATLAB's *ga* function [3] minimizes the expected time:

- **Objective:** Handle to *evaluate_strategy*.
- **Variables & Type:** 2 integer variables (K, M) .
- **Bounds & Constraints:** Enforces $1 \leq K < M \leq \text{NumLaps}$.
- **Options:** ** Standard GA settings with convergence plots enabled.

E. Detailed Simulation (*simulate_detailed_race.m*)

Runs one simulation using the optimal (K, M) found, logging lap-by-lap data (time, tire, SC status, etc.) for visualization.

III. RESULTS AND DISCUSSION

The framework was executed using placeholder tire models due to data fitting limitations. The optimization successfully identified optimal (K, M) strategies for each track based on these models and the track-specific parameters.

A. Optimal Strategies and Cross-Track Comparison

The GA optimization yielded distinct optimal (K, M) strategies, summarized in Table I and Fig. 2.

TABLE I: Optimal (K, M) Strategy and Expected Time per Track (Using Placeholder Tire Models)

Track	Optimal K	Optimal M	Min Exp. Time (s)	Comp. Time (s)
Australian GP (2025)	32	42	4821.53	115.52
Monaco GP (2024)	52	69	6528.24	156.87
Japanese GP (2025)	27	37	4400.08	113.36
Bahrain GP (2025)	29	41	4741.30	118.71

Analysis: Despite using simplified tire models, the results clearly demonstrate the optimization framework’s ability to adapt the strategy to track characteristics.

- **Monaco GP (High SC Prob, Long Race):** The high SC likelihood (0.25/lap) dominates the strategy, pushing the SC threshold ($K = 52$) and mandatory stop ($M = 69$) very late. The potential gain from a cheap SC stop outweighs the tire degradation penalty (as modeled by placeholders) over a long race (78 laps). The SC waiting window ($M - K$) is 17 laps, the largest I saw.
- **Japanese GP (Low SC Prob, Shorter Race):** With low SC probability (0.04/lap) and the shortest race length (53 laps), the strategy is the most aggressive, favoring an early SC threshold ($K = 27$) and the earliest mandatory stop ($M = 37$). There’s minimal incentive to wait for an unlikely SC. The waiting window is 10 laps.
- **Bahrain GP (Low SC Prob, Moderate Race):** Similar low SC probability (0.03/lap) but slightly longer race (57 laps) compared to Japan. This results in a slightly later SC threshold ($K = 29$) and mandatory stop ($M = 41$). The waiting window is 12 laps.
- **Australian GP (Moderate SC Prob):** The strategy ($K = 32, M = 42$) is intermediate, reflecting the moderate SC probability (0.06) and race length (58 laps). The waiting window (10 laps) is similar to Japan’s, suggesting the moderate SC chance doesn’t drastically push the window compared to the low-probability tracks in this model.

These relative differences strongly suggest that SC probability and race length are primary drivers of the optimal (K, M) window in this model, influencing the trade-off between immediate tire performance and waiting for a stochastic advantage. Higher SC probability clearly encourages larger values for both K and M.

B. Optimization Performance and Landscape

The GA convergence plots (Fig. 3) show consistent convergence to a minimum expected time value for each track within the specified generations.

The local heatmap (Fig. 5-7) reveals the sensitivity near the optimum. For the Australian GP (based on placeholder models), the heatmap around (33, 42) shows a defined minimum region. Moving K earlier or M later increases the expected time noticeably, suggesting the optimum is reasonably well-defined even with the simplified tire models. For the Monaco GP, we can see that

M

was not as constrained as K was because the frequency of SC is so much higher. Bahrain’s Heatmap revealed that SC pit threshold lap tens to be higher than what we got for optimal race time.

C. Strategy Dynamics and Uncertainty

The detailed simulation plot (Fig. 8) provides a snapshot of one race outcome. This run for Australia ($K=33, M=42$) shows the car starting on the placeholder Medium, pitting on lap $M=42$ for placeholder Hards as no SC occurred at or after lap $K=33$. Lap times follow the placeholder degradation curves. As we can see the laps, get a lot slower when the safety car is active, and our lap time gradually increases after pitting.

The result distribution histograms (Fig. 9 - 12) illustrate the impact of the stochastic SC events on the final race time. Monaco’s distribution (Fig. 10) is significantly wider than Bahrain’s (Fig. 12), reflecting the much higher SC probability and its potential to drastically alter race time. The optimal strategy minimizes the *average* time, but individual outcomes can vary substantially, especially on high-SC tracks.

D. Discussion and Limitations

The framework successfully optimizes the (K, M) strategy based on track parameters, demonstrating a core principle of adaptive strategy under uncertainty. The GA effectively handles the simulation-based, non-convex optimization task.

The most significant limitation is the inability to fit reliable data-driven tire models from the provided CSVs using the current single-stint approach. The filtering required to isolate clean racing laps often leaves too few data points. This means the absolute optimized times are unrealistic, and the analysis relies on the relative strategy shifts driven by track parameters (laps, SC prob) interacting with the placeholder tire models. Future work must address this, potentially by:

- Aggregating data across multiple stints or drivers for a given compound/track.
- Incorporating fuel load corrections.
- Using more advanced time-series or physics-based tire models.
- Targeting sessions/drivers known to perform long, clean runs.

Other limitations remain: single-car focus, simplified SC mechanics, and lack of other real-world factors (weather, traffic, etc.).

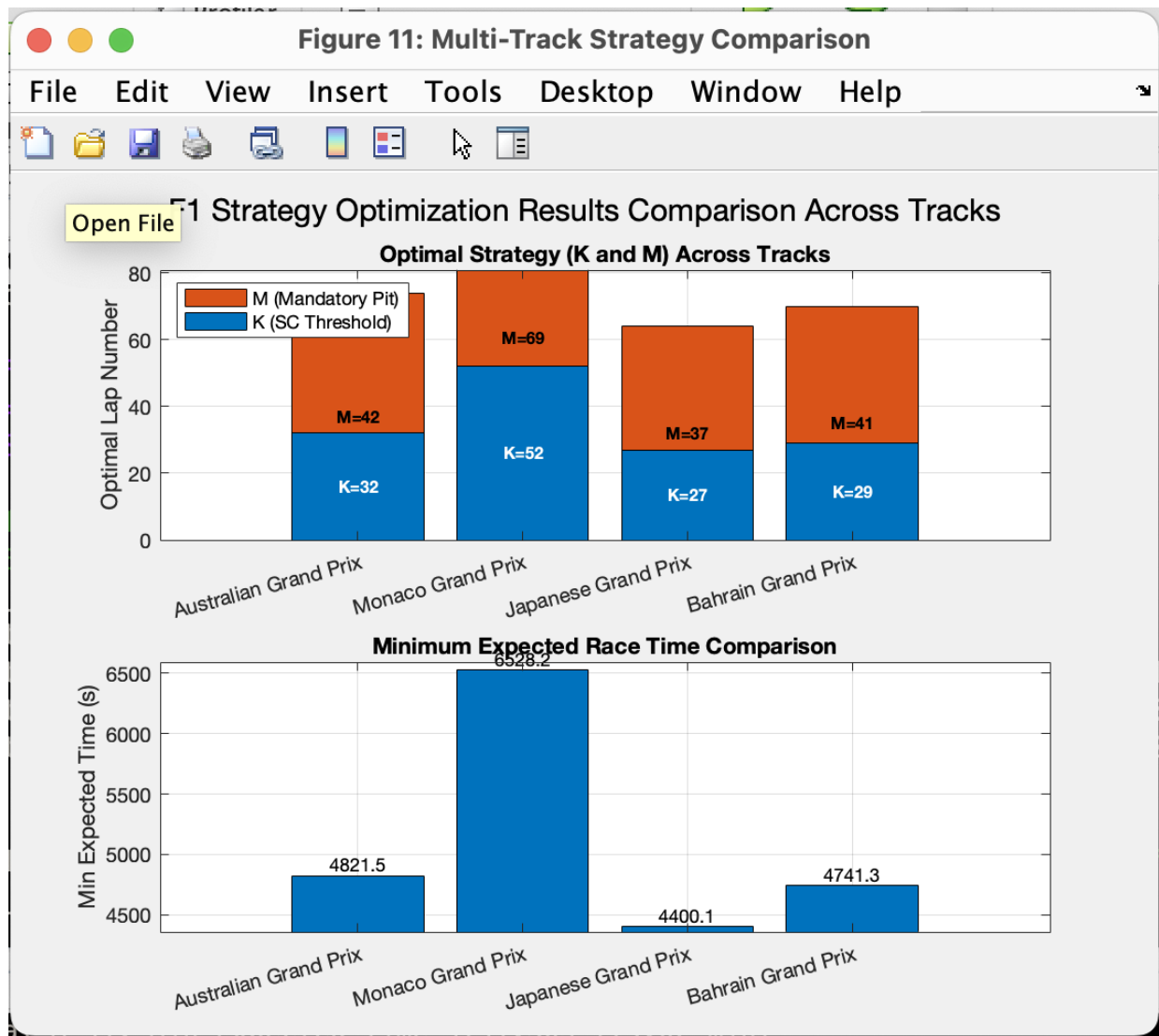


Fig. 2: Comparison of optimal strategies (K , M) and minimum expected race times across analyzed tracks (based on placeholder tire models).

From an ESE 415 perspective, the project highlights the practical challenges of applying optimization. While the theory often assumes well-behaved functions, real problems require dealing with noisy or sparse data, simulation-based objectives, and choosing appropriate algorithms (like GA) when standard assumptions fail.

IV. CONCLUSION

This project developed and applied an optimization framework using MATLAB and its Global Optimization Toolbox to determine optimal dynamic F1 pit stop strategies (K , M) under Safety Car uncertainty. The objective was minimizing the expected total race time, evaluated via Monte Carlo simulation incorporating track-specific parameters. The optimization depended solely on placeholder models since fitting tire degradation models from the accessible lap-by-lap data

for single stints didn't give us enough data to filter through for robust models

The Genetic Algorithm effectively found ideal (K , M) strategies for several tracks (Australia, Monaco, Japan, Bahrain) despite this restriction. The results unequivocally show that the best approach is quite sensitive to track conditions, especially the probability of Safety Car deployment and overall race length. Higher SC probabilities clearly favor later pit windows (larger K and M), so quantifying the strategic value placed on perhaps obtaining from a cheap SC stop.

Using suitable tools (Monte Carlo, GA), the project essentially shows the formulation and solution of a simulation-based stochastic optimization problem. Important visualizations show the terrain of optimization, result uncertainty, and the relative variations in ideal strategies across tracks. Although depending on placeholder tire models reduces the predictive accuracy of absolute race times, the framework

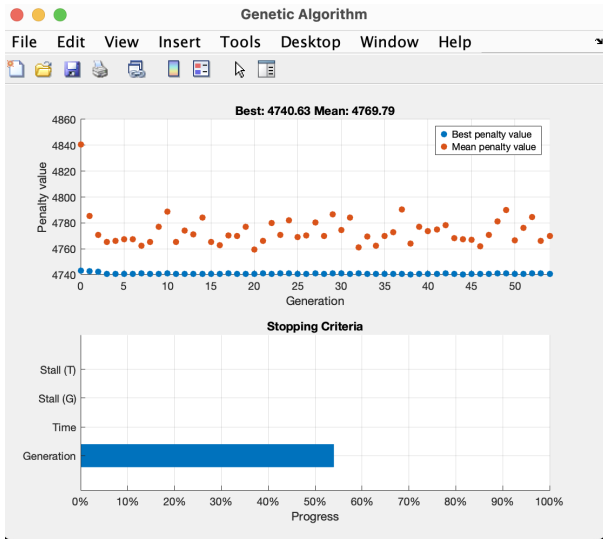


Fig. 3: Genetic Algorithm convergence plot (Best Fitness vs. Generation) for Australian GP optimization.

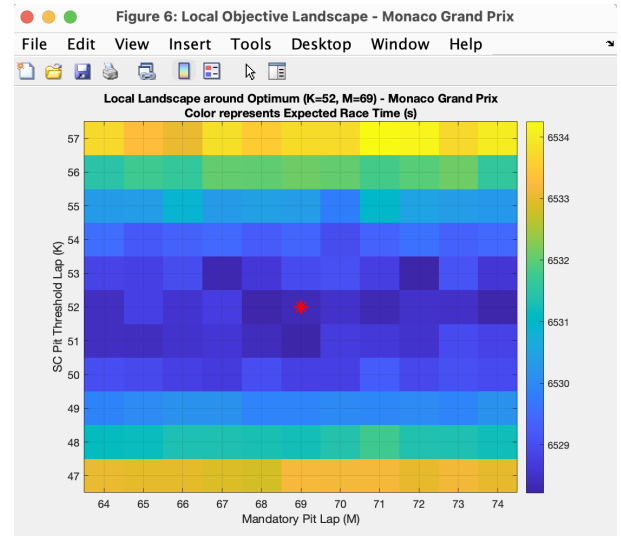


Fig. 5: Local objective function landscape around the optimum for the Monaco GP (based on placeholder tire models).

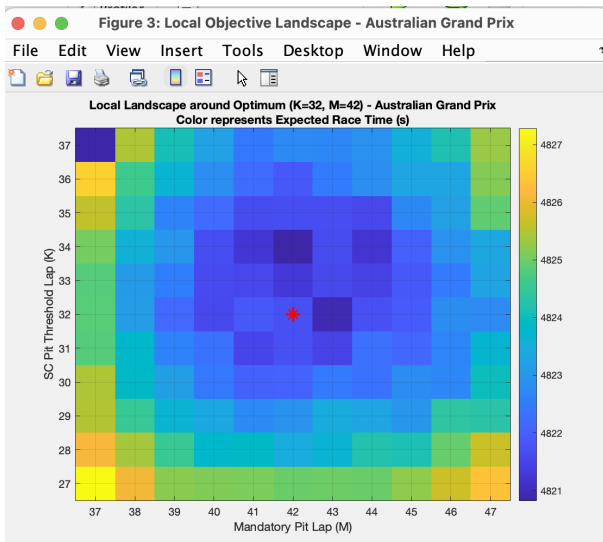


Fig. 4: Local objective function landscape around the optimum (K=32, M=42) for the Australian GP (based on placeholder tire models).

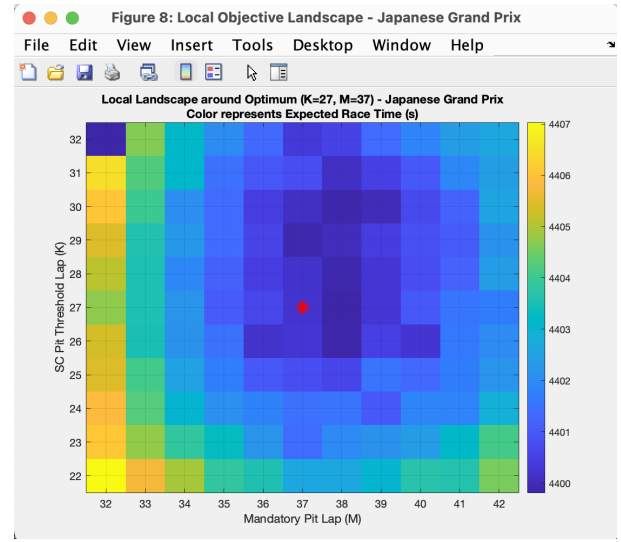


Fig. 6: Local objective function landscape around the optimum for the Japanese GP (based on placeholder tire models).

effectively captures the fundamental strategic trade-offs and offers insightful analysis of how to modify strategy to fit environmental uncertainty, so accomplishing the objective of using optimization ideas on a challenging, dynamic system. Strong data aggregation and modeling methods for tire degradation should take front stage in future work in order to improve simulation fidelity.

REFERENCES

- [1] The Oehrly, "Fast-F1: Wrapper for F1 data," GitHub Repository, 2024. [Online]. Available: <https://github.com/theOehrly/Fast-F1>. [Accessed: Apr. 30, 2025].
- [2] U. S. Kamilov, "ESE 415 Optimization Notes," Course Materials, Washington University in St. Louis, St. Louis, MO, 2023. [Accessed: Apr. 30, 2025].

- [3] MathWorks, "Genetic Algorithm - MATLAB ga," MATLAB Documentation, Natick, MA. [Online]. Available: <https://www.mathworks.com/help/gads/ga.html>. [Accessed: Apr. 30, 2025].
- [4] M. Dodds, "Inside F1 strategy: How data analysis and simulation shape race-winning calls," Motorsport.com, Aug. 15, 2022. [Online]. Available: <https://www.motorsport.com/f1/news/inside-f1-strategy-how-data-analysis-and-simulation-shape-race-winning-calls/10353058/>. [Accessed: Apr. 30, 2025].

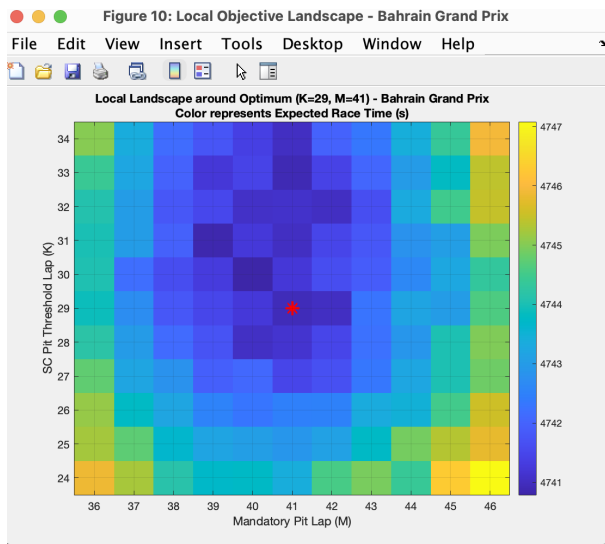


Fig. 7: Local objective function landscape around the optimum for the Bahrain GP (based on placeholder tire models).

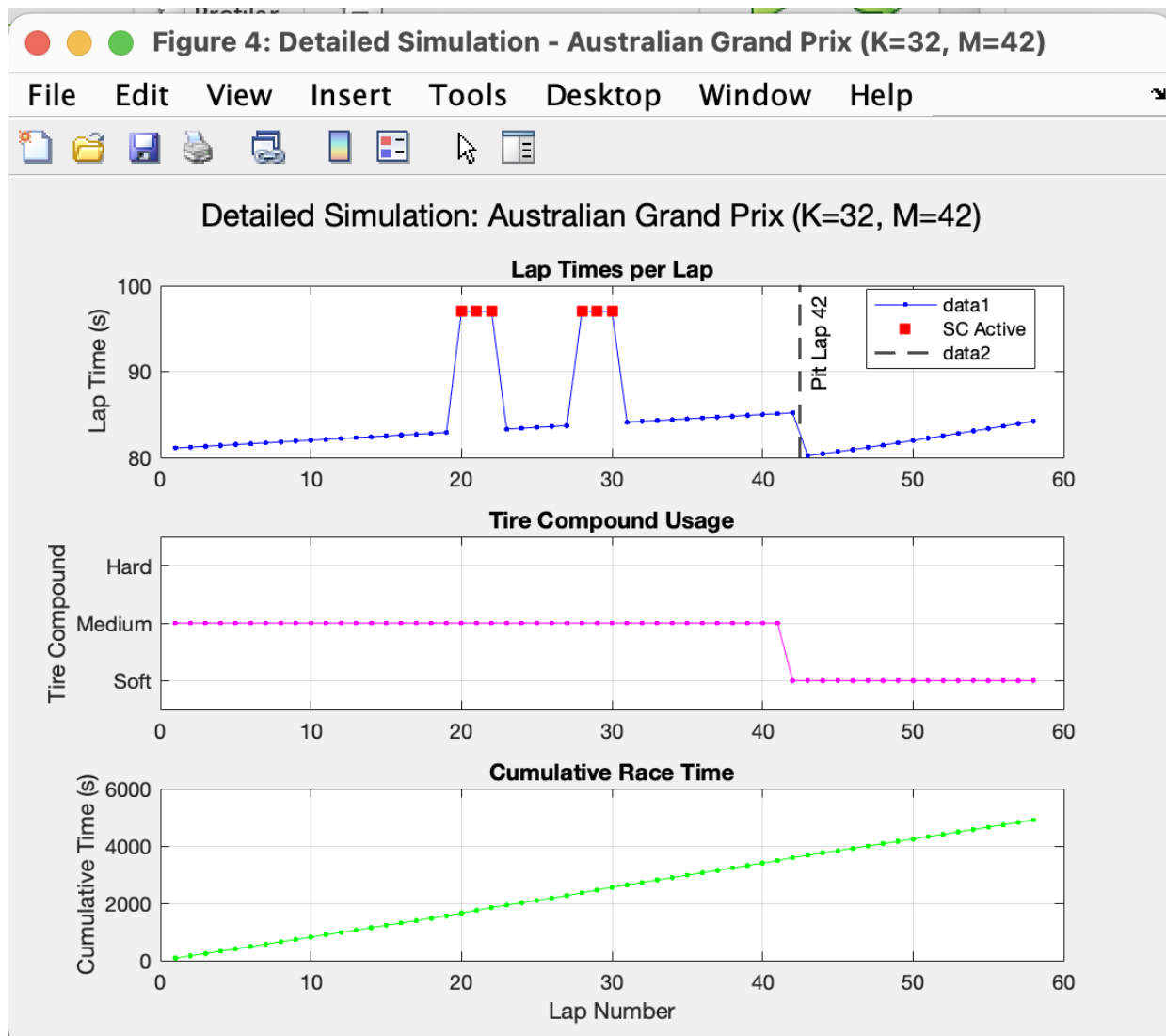


Fig. 8: Detailed lap-by-lap simulation results for one run using the optimal strategy ($K=33$, $M=42$) at the Australian GP (based on placeholder tire models).

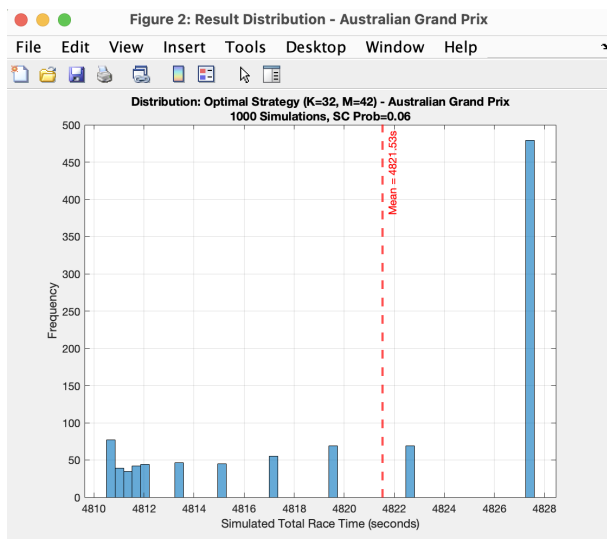


Fig. 9: Result distribution for optimal strategy at Australian GP (placeholder models).

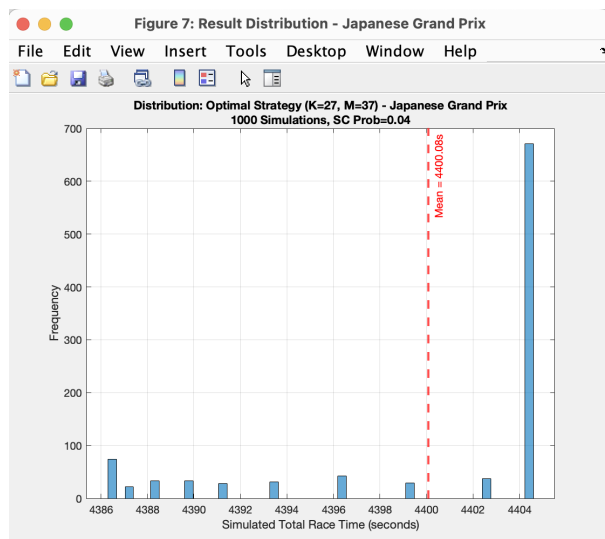


Fig. 11: Result distribution for optimal strategy at Japanese GP (placeholder models).

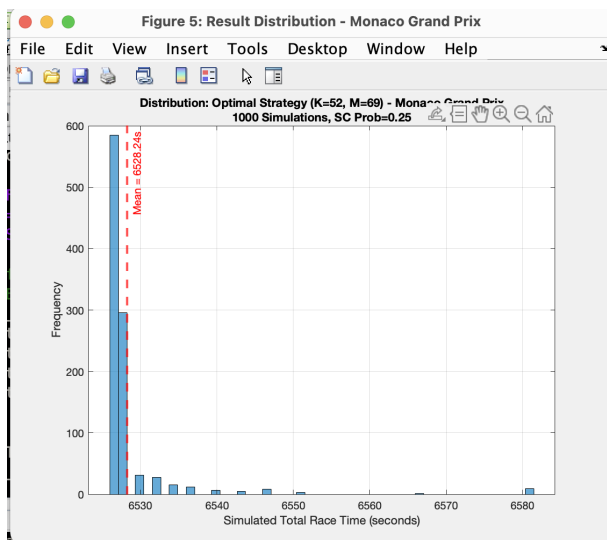


Fig. 10: Result distribution for optimal strategy at Monaco GP (placeholder models).

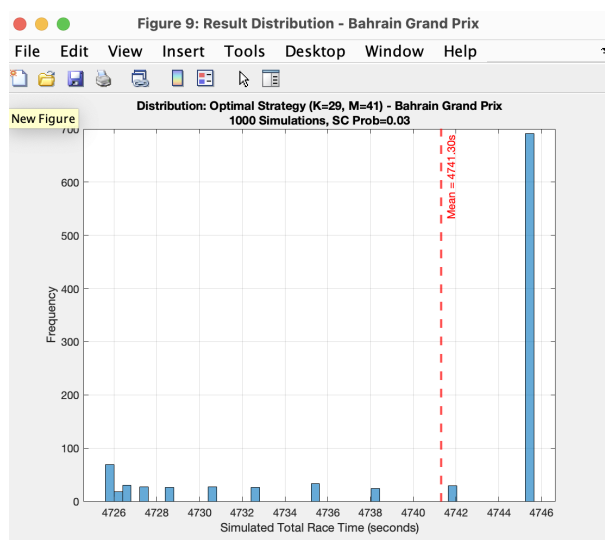


Fig. 12: Result distribution for optimal strategy at Bahrain GP (placeholder models).

Selected core functions used in the simulation and optimization framework.

A. main_optimize_strategy.m

```

1  % F1 Race Strategy Optimization for multiple tracks
2
3  clear; clc; close all; % Start with a clean workspace
4
5  fprintf('F1 Race Strategy Optimization - MATLAB\n');
6  fprintf('=====\n');
7  ;
8  fprintf('Start Time: %s\n\n', datestr(now));
9
10 % Configuration
11 % SELECT TRACKS TO RUN
12 tracks_to_analyze = { ...
13     struct('Name', 'Australian Grand Prix', 'Folder',
14           'Aus', 'Year', 2025, 'Session', 'Practice 2'), ...
15     struct('Name', 'Monaco Grand Prix', 'Folder',
16           'Monaco', 'Year', 2024, 'Session', 'Race'), ...
17     struct('Name', 'Japanese Grand Prix', 'Folder',
18           'Japan', 'Year', 2025, 'Session', 'Practice 3'), ...
19     struct('Name', 'Bahrain Grand Prix', 'Folder',
20           'Bahrain', 'Year', 2025, 'Session', 'Race') ...
21 };
22
23 USE_UNCERTAINTY = true; % true: Optimize (K,M) using
24   GA | false: Optimize fixed strategy (Grid Search)
25
26 BASE_DATA_FOLDER = 'data'; % Base folder containing
27   track subfolders
28
29 %Data Fitting Configuration
30 TARGET_DRIVER = 'VER';
31 TARGET_COMPOUND = 'MEDIUM';
32 TARGET_STINT = 1;
33
34 %Visualization Configuration
35 RUN_DETAILED_SIM_FOR_TRACK = 'Australian Grand Prix'
36   ; % Choose one track for lap-by-lap plots
37 LOCAL_HEATMAP_RANGE = 5; % +/- laps around optimum
38   for local heatmap
39
40 % Storage for Comparative Results
41 num_tracks = length(tracks_to_analyze);
42 comparison_results = struct('TrackName', cell(1,
43   num_tracks), ...
44   'OptimalK', NaN(1,
45   num_tracks), ...
46   'OptimalM', NaN(1,
47   num_tracks), ...
48   'MinExpectedTime', NaN
49   (1, num_tracks), ...
50   'ComputationTime', NaN
51   (1, num_tracks));
52
53 % Loop Through Each Track
54 for i_track = 1:num_tracks
55     current_track = tracks_to_analyze{i_track};
56     fprintf('\n\n==== Processing Track: %s (%d)
57     =====\n', current_track.Name, current_track.Year
58     );
59
60 % Construct file path for data fitting
61 track_data_folder = fullfile(BASE_DATA_FOLDER,
62   current_track.Folder);

```

```

46 csv_filename = sprintf('%d-%s-%s.csv',
47   current_track.Year, current_track.Name,
48   current_track.Session);
49 csv_filepath = fullfile(track_data_folder,
50   csv_filename);
51
52 %Load Parameters and Models
53 fprintf('--- Loading Parameters and Models for %
54 s ---\n', current_track.Name);
55 race_params = define_race_parameters(
56   current_track.Year, current_track.Name);
57 try
58     tire_models = define_data_driven_tire_models
59   (csv_filepath, TARGET_DRIVER, TARGET_COMPOUND,
60   TARGET_STINT);
61 catch ME
62     warning('Track %s: Could not load data-
63   driven tire models. Using placeholder models.
64   Error: %s', current_track.Name, ME.message);
65     tire_models = define_placeholder_tire_models
66   ();
67 end
68
69 % Validate Inputs
70 if isempty(tire_models) || isempty(race_params),
71     error('Track %s: Failed to load parameters or
72   tire models.', current_track.Name); end
73 if race_params.num_laps <= 0, error('Track %s:
74   Number of laps must be positive.', current_track
75   .Name); end
76 fprintf('
77 -----\n\n');
78
79 % Run Optimization
80 fprintf('--- Starting Optimization for %s (Using
81   Genetic Algorithm) ---\n', current_track.Name);
82 tic;
83 optimal_strategy = []; min_objective_value = Inf
84 ; % Initialize
85 try
86     [optimal_strategy, min_objective_value, ~,
87     ~, ~] = ...
88     find_optimal_strategy(race_params,
89   tire_models, USE_UNCERTAINTY);
90 catch ME
91     fprintf('\nError during optimization for %s:
92   %s\n', current_track.Name, ME.message);
93     disp(ME.getReport());
94 end
95 elapsed_time = toc;
96 fprintf('--- Optimization Finished for %s ---\n'
97 , current_track.Name);
98
99 % Store Results
100 comparison_results(i_track).TrackName =
101   current_track.Name;
102 comparison_results(i_track).ComputationTime =
103   elapsed_time;
104 if ~isempty(optimal_strategy) && isfinite(
105   min_objective_value)
106     comparison_results(i_track).OptimalK =
107   optimal_strategy(1);
108     comparison_results(i_track).OptimalM =
109   optimal_strategy(2);
110     comparison_results(i_track).MinExpectedTime
111   = min_objective_value;
112     fprintf('\n--- Optimal Strategy Found for %s
113   ---\n', current_track.Name);
114     fprintf(' Optimal K=%d, Optimal M=%d, Min
115   Expected Time=%.2fs (%.2f min)\n',
116   optimal_strategy(1), optimal_strategy(2),
117   min_objective_value, min_objective_value/60);

```

```

87         fprintf(' Optimization Time: %.2f seconds\n', 126
, elapsed_time);
88
89         K_opt = optimal_strategy(1); M_opt = 127
optimal_strategy(2);
90
91         % Distribution Plot (using evaluate_strategy 128
)
92         fprintf('\n Generating Result Distribution 129
Plot for %s...\n', current_track.Name);
93         try
94             [~, simulation_times_hist] = 130
evaluate_strategy([K_opt, M_opt], race_params,
tire_models, true, 'dynamic_KM'); % Assumes
evaluate_strategy returns times
95             figure('Name', sprintf('Result 131
Distribution - %s', current_track.Name));
96             histogram(simulation_times_hist, 50); 132
hold on;
97             xline(min_objective_value, 'r--', ' 133
LineWidth', 2, 'Label', sprintf('Mean = %.2fs',
min_objective_value));
98             xlabel('Simulated Total Race Time ( 134
seconds)'); ylabel('Frequency');
99             title(sprintf('Distribution: Optimal 135
Strategy (K=%d, M=%d) - %s', K_opt, M_opt,
current_track.Name), ...
100             sprintf('%d Simulations, SC Prob 136
= %.2f', length(simulation_times_hist),
race_params.safety_car_probability_per_lap));
101             grid on; hold off;
102             fprintf(' Visual: Displayed Result 137
Distribution Histogram.\n');
103             catch ME_hist
104                 warning('Could not generate distribution 138
plot for %s: %s', current_track.Name, ME_hist.
message);
105             end
106
107             % Local Heatmap around Optimum 139
fprintf('\n Generating Local Heatmap for %s
...\n', current_track.Name);
108             try
109                 k_range = max(1, K_opt - 140
LOCAL_HEATMAP_RANGE) : min(race_params.num_laps
- 1, K_opt + LOCAL_HEATMAP_RANGE);
110                 m_range = max(min(k_range)+1, M_opt - 141
LOCAL_HEATMAP_RANGE) : min(race_params.num_laps,
M_opt + LOCAL_HEATMAP_RANGE);
111                 local_results = NaN(length(k_range), 142
length(m_range));
112                 fprintf(' Evaluating %d points for local 143
heatmap...\n', length(k_range)*length(m_range))
;
113                 for i_k_local = 1:length(k_range) 144
k_local = k_range(i_k_local);
114                 for i_m_local = 1:length(m_range) 145
m_local = m_range(i_m_local);
115                 if k_local < m_local % Only 146
evaluate valid strategies
116                     local_results(i_k_local, 147
i_m_local) = evaluate_strategy([k_local, m_local
], race_params, tire_models, true, 'dynamic_KM')
;
117                     end
118                 end
119                 figure('Name', sprintf('Local Objective 148
Landscape - %s', current_track.Name));
120                 imagesc(m_range, k_range, local_results) 149
; colorbar; axis xy; grid on;
121                 hold on; plot(M_opt, K_opt, 'r*', ' 150
MarkerSize', 12, 'LineWidth', 2); hold off;
122
123                 xlabel('Mandatory Pit Lap (M)'); ylabel( 151
'SC Pit Threshold Lap (K)');
124                 title(sprintf('Local Landscape around 152
Optimum (K=%d, M=%d) - %s', K_opt, M_opt,
current_track.Name), ...
125                 'Color represents Expected Race 153
Time (s)'));
126                 set(gca, 'FontSize', 10);
127                 fprintf(' Visual: Displayed Local 154
Heatmap.\n');
128                 catch ME_heatmap
129                     warning('Could not generate local 155
heatmap for %s: %s', current_track.Name,
ME_heatmap.message);
130                 end
131
132                 % Detailed Lap-by-Lap Simulation Plot (Only 156
for specified track)
133                 if strcmpi(current_track.Name, 157
RUN_DETAILED_SIM_FOR_TRACK)
134                     fprintf('\n Generating Detailed Lap 158
Plot for %s...\n', current_track.Name);
135                     try
136                         % Run one detailed simulation, 159
logging data
137                         [lap_data_table] = 160
simulate_detailed_race([K_opt, M_opt],
race_params, tire_models);
138
139                         % Create Plots
140                         figure('Name', sprintf('Detailed 161
Simulation - %s (K=%d, M=%d)', current_track.
Name, K_opt, M_opt));
141
142                         % Lap Times Plot
143                         subplot(3, 1, 1);
144                         plot(lap_data_table.LapNumber, 162
lap_data_table.LapTime, 'b.-'); hold on;
145                         sc_laps = lap_data_table.LapNumber( 163
lap_data_table.SC_Active);
146                         plot(sc_laps, lap_data_table. 164
LapTime(lap_data_table.SC_Active), 'rs', '
MarkerFaceColor', 'r', 'DisplayName', 'SC Active
');
147                         pit_lap = lap_data_table.LapNumber( 165
lap_data_table.PitStop);
148                         if ~isempty(pit_lap)
149                             xline(pit_lap + 0.5, 'k--', ' 166
LineWidth', 1.5, 'Label', sprintf('Pit Lap %d',
pit_lap));
150                         end
151                         ylabel('Lap Time (s)'); title('Lap 167
Times per Lap'); grid on; legend('show', '
Location', 'best');
152
153                         % Tire Compound Plot
154                         subplot(3, 1, 2);
155                         compound_indices = lap_data_table. 168
CompoundIndex;
156                         tire_names_laps = {tire_models(
compound_indices).name};
157                         plot(lap_data_table.LapNumber, 169
compound_indices, 'm.-');
158                         yticks(1:length(tire_models));
159                         yticklabels({tire_models.name});
160                         ylabel('Tire Compound'); grid on;
161                         ylim([0.5, length(tire_models)+0.5]);
162                         title('Tire Compound Usage');
163
164                         % Cumulative Time Plot
165                         subplot(3, 1, 3);
166                         plot(lap_data_table.LapNumber, 170
lap_data_table.CumulativeTime, 'g.-');

```

```

169         xlabel('Lap Number'); ylabel('
Cumulative Time (s)');
170         title('Cumulative Race Time'); grid
on;
171
172         sgtitle(sprintf('Detailed
Simulation: %s (K=%d, M=%d)', current_track.Name
, K_opt, M_opt));
173         fprintf(' Visual: Displayed
Detailed Lap-by-Lap Plots.\n');
174
175         catch ME_detail
176             warning('Could not generate
detailed simulation plot for %s: %s',
current_track.Name, ME_detail.message);
177         end
178         end % End detailed plot check
179
180     else
181         fprintf('No valid strategy found or
optimization failed for %s.\n', current_track.
Name);
182     end
183     fprintf('
===== \n');
184
185 end % End loop through tracks
186
187 % Final Visuals
188 fprintf('\n--- Generating Final Multi-Track
Comparison Plot ---\n');
189
190 try
191     figure('Name', 'Multi-Track Strategy Comparison'
);
192     valid_results_idx = ~isnan([comparison_results.
MinExpectedTime]); % Filter out tracks where
optimization failed
193     track_names = {comparison_results(
valid_results_idx).TrackName};
194     optimal_Ks = [comparison_results(
valid_results_idx).OptimalK];
195     optimal_Ms = [comparison_results(
valid_results_idx).OptimalM];
196     min_times = [comparison_results(
valid_results_idx).MinExpectedTime];
197     num_valid_tracks = length(track_names);
198
199     if num_valid_tracks == 0
200         error('No valid optimization results to plot
. ');
201     end
202
203     % Subplot 1: Optimal K and M
204     subplot(2, 1, 1);
205     bar_data_KM = [optimal_Ks', optimal_Ms']; % K
first, then M
206     b_KM = bar(bar_data_KM, 'stacked');
207     set(gca, 'xticklabel', track_names, '
XTickLabelRotation', 15); % Rotate labels if
needed
208     ylabel('Optimal Lap Number');
209     title('Optimal Strategy (K and M) Across Tracks'
);
210     legend({'K (SC Threshold)', 'M (Mandatory Pit)'
}, 'Location', 'NorthWest'); % Correct legend
order if needed
211     grid on;
212     ylim([0, max(optimal_Ms)*1.1 + 5]);
213
214     % Add text labels for K and M values
215     for i = 1:num_valid_tracks
216         k_val = optimal_Ks(i);
217         m_val = optimal_Ms(i);
218         if ~isnan(k_val)
219             text(i, k_val/2, sprintf('K=%d', k_val)
, ...
'HorizontalAlignment', 'center', '
VerticalAlignment', 'middle', 'Color', 'w', '
FontSize', 8, 'FontWeight', 'bold');
220         end
221         if ~isnan(m_val) && ~isnan(k_val)
222             text(i, k_val + (m_val-k_val)/2,
sprintf('M=%d', m_val), ... % Position text in
the M part
'HorizontalAlignment', 'center', '
VerticalAlignment', 'middle', 'Color', 'k', '
FontSize', 8, 'FontWeight', 'bold');
223         elseif ~isnan(m_val)
224             text(i, m_val/2, sprintf('M=%d', m_val)
), ...
'HorizontalAlignment', 'center', '
VerticalAlignment', 'middle', 'Color', 'k', '
FontSize', 8, 'FontWeight', 'bold');
225         end
226     end
227     b_KM(1).FaceColor = [0 0.4470 0.7410]; % Blue
for K
228     b_KM(2).FaceColor = [0.8500 0.3250 0.0980]; %
Orange for M-K part
229
230     % Subplot 2: Minimum Expected Time
231     subplot(2, 1, 2);
232     b_time = bar(min_times);
233     set(gca, 'xticklabel', track_names, '
XTickLabelRotation', 15);
234     ylabel('Min Expected Time (s)');
235     title('Minimum Expected Race Time Comparison');
236     grid on;
237     if ~isempty(min_times)
238         ylim([min(min_times)*0.99, max(min_times)
*1.01]);
239         xtips_time = b_time.XEndPoints;
240         ytips_time = b_time.YEndPoints;
241         labels_time = string(round(b_time.YData,1));
242         text(xtips_time, ytips_time, labels_time, '
HorizontalAlignment', 'center', ...
'VerticalAlignment', 'bottom', 'FontSize'
, 9);
243     end
244
245     sgtitle('F1 Strategy Optimization Results
Comparison Across Tracks'); % Overall title
246
247     fprintf(' Visual: Displayed Multi-Track
Comparison Plot.\n');
248     %
249
250     catch ME_compare_final
251         warning('Could not generate final comparison
plot: %s', ME_compare_final.message);
252     end
253
254     fprintf('\n--- End of Multi-Track Simulation ---\n'
);
255
256
257

```

Listing 1: Main script orchestrating the multi-track analysis.

B. find_optimal_strategy.m

```

1 % --- File: find_optimal_strategy.m (Modified -
km_constraint removed) ---
2 function [best_strategy, min_time, results_matrix,
possible_K, possible_M] = find_optimal_strategy(
race_params, tire_models, use_uncertainty)
3 % Finds the optimal strategy.
4 % If use_uncertainty=true, searches over (K,M)
dynamic strategy using GA.

```

```

5 % If use_uncertainty=false, searches over fixed
6 1/2-stop strategies (example grid search).
7
8 disp('Finding Optimal Strategy...');
9 min_time = Inf;
10 best_strategy = [];
11 results_matrix = []; % GA doesn't naturally
12 produce this for heatmap
13 possible_K = []; % GA doesn't naturally
14 produce this
15 possible_M = []; % GA doesn't naturally
16 produce this
17
18 if use_uncertainty
19 % --- Setup Genetic Algorithm for (K, M) ---
20 strategy_type = 'dynamic_KM';
21 disp('Searching over (K, M) dynamic
22 strategies using Genetic Algorithm...');
23
24 % Define the objective function handle for
25 GA
26 % It takes a vector [K, M] and returns the
27 expected time
28 % Make sure evaluate_strategy is accessible
29 (either in the same folder or on MATLAB path)
30 objectiveFunction = @(vars)
31 evaluate_strategy(vars, race_params, tire_models
32 , true, strategy_type);
33
34 numberOfVariables = 2; % We are optimizing K
35 and M
36
37 % Define bounds for K and M
38 min_M_bound = 10; % Lower bound for M
39 max_M_bound = race_params.num_laps - 5; %
40 Upper bound for M
41 min_K_bound = 1; % Lower bound for K
42 max_K_bound = max_M_bound - 1; % Upper bound
43 for K (must be < M)
44
45 lb = [min_K_bound, min_M_bound]; % Lower
46 bounds [K, M]
47 ub = [max_K_bound, max_M_bound]; % Upper
48 bounds [K, M]
49
50 % Specify that both variables are integers
51 IntCon = [1, 2];
52
53 % Define the non-linear constraint K < M
54 using the separate function file
55 % Make sure km_constraint.m is in the same
56 folder or on MATLAB path
57 constraintFunction = @km_constraint;
58
59 % Set GA options (can be tuned)
60 % Ensure you have the Global Optimization
61 Toolbox installed
62 options = optimoptions('ga', ...
63 'Display', 'iter', ... % Show progress
64 ('off', 'iter', 'final')
65 'PopulationSize', 50, ... % Number of
66 individuals per generation
67 'MaxGenerations', 100, ... % Number of
68 iterations (adjust as needed)
69 'PlotFcn', {@gaplotbestf,
70 @gaplotstopping}, ... % Optional: plot progress
71 'UseParallel', false); % Set to true if
72 Parallel Computing Toolbox is available and
73 desired
74
75 % --- Run the Genetic Algorithm ---
76 fprintf('Running Genetic Algorithm...\n');
77 [best_vars, min_time] = ga(objectiveFunction
78 , numberOfVariables, ...
79
80 lb, ub, ...
81
82 constraintFunction, IntCon, options);
83
84 % Assign the best strategy found by GA
85 best_strategy = round(best_vars); % Ensure K
86 , M are integers
87
88 fprintf('>>> GA finished. Best strategy
89 found (K=%d, M=%d): Expected Time = %.2f\n', ...
90 best_strategy(1), best_strategy(2),
91 min_time);
92 % Note: results_matrix, possible_K,
93 possible_M are not generated by GA
94
95 else
96 % --- Search over Fixed Strategies (
97 Deterministic Example: 1-stop) ---
98 % (Keep your existing grid search code here
99 for the deterministic case)
100 strategy_type = 'fixed';
101 disp('Searching over fixed 1-stop strategies
102 (deterministic)...');
103 num_laps = race_params.num_laps;
104 min_pit_lap = 5;
105 max_pit_lap = num_laps - 5;
106 start_compound_idx = race_params.
107 start_compound_idx;
108 available_compounds = 1:length(tire_models);
109 next_compounds = available_compounds(
110 available_compounds ~= start_compound_idx);
111
112 for pit_lap = min_pit_lap:max_pit_lap
113 stint1_laps = pit_lap;
114 stint2_laps = num_laps - pit_lap;
115 if stint1_laps <= 0 || stint2_laps <= 0,
116 continue; end
117
118 for c2 = next_compounds % Iterate
119 through valid next compounds
120 strategy_params = [
121 start_compound_idx, stint1_laps; c2, stint2_laps
122 ]; % Fixed strategy matrix
123 current_time = evaluate_strategy(
124 strategy_params, race_params, tire_models, false
125 , strategy_type);
126
127 if current_time < min_time
128 min_time = current_time;
129 best_strategy = strategy_params;
130 % Best strategy is the matrix
131 fprintf('>>> New best fixed (Pit
132 Lap %d, %s -> %s): Time = %.2f\n', ...
133 pit_lap, tire_models(
134 start_compound_idx).name, tire_models(c2).name,
135 min_time);
136 end
137 end
138 fprintf('Finished evaluating fixed
139 strategies.\n');
140 % Add logic for 2-stops etc. if needed for
141 deterministic case
142 end
143
144 disp('Optimization Finished.');
```

Listing 2: Function implementing the Genetic Algorithm optimization.

C. evaluate_strategy.m

```

1 function [expected_time, simulation_times] =
2     evaluate_strategy(strategy_params, race_params,
3         tire_models, use_uncertainty, strategy_type)
4     % Evaluates a strategy based on type.
5     % K = Pit if SC occurs >= lap K. M = Mandatory
6     % pit lap if no SC pit occurred.
7     simulation_times = []; %
8     % data type validation
9     if ~isstruct(race_params) || ~isstruct(
10         tire_models)
11         error('race_params and tire_models must be
12             structs.');
```

13 end

14 if ~islogical(use_uncertainty)

15 error('use_uncertainty must be logical (true

16 /false).');

17 end

18 if strcmp(strategy_type, 'fixed')

19 if use_uncertainty

20 warning('Evaluating fixed strategy

21 deterministically, ignoring uncertainty flag.');

22 % Or: you could adapt simulate_race to

23 handle uncertainty for fixed plans too

24 end

25 % Call simulate_race_fixed

26 expected_time = simulate_race_fixed(

27 strategy_params, race_params, tire_models);

28 elseif strcmp(strategy_type, 'dynamic_KM')

29 if ~use_uncertainty

30 error('Dynamic KM strategy requires

31 uncertainty simulation.');

32 end

33 % Validate strategy_params format

34 if ~isnumeric(strategy_params) || numel(

35 strategy_params) ~= 2

36 error('For dynamic_KM, strategy_params

37 must be a 1x2 numeric vector [K, M].');

38 end

39 K = round(strategy_params(1)); % Ensure

40 integer K

41 M = round(strategy_params(2)); % Ensure

42 integer M

43 % Validate K and M values

44 if K < 1 || M < 1 || M > race_params.

45 num_laps

46 warning('K or M value is out of

47 reasonable bounds. K=%d, M=%d', K, M);

48 expected_time = Inf; % Penalize invalid

49 K or M

50 return;

51 end

52 if K >= M

53 expected_time = Inf; % Invalid strategy:

54 SC threshold >= Mandatory pit lap

55 return;

56 end

57 start_compound_idx = race_params.

58 start_compound_idx; % Get start compound

59 % Determine the compound index for the

60 second stint based on rules

61 available_compounds = 1:length(tire_models);

62 % Ensure start_compound_idx is valid

63 if start_compound_idx < 1 ||

64 start_compound_idx > length(tire_models)

65 error('Invalid start_compound_idx: %d',

66 start_compound_idx);

67 end

68 next_compound_idx = find(available_compounds

69 ~= start_compound_idx, 1, 'first');

70 if isempty(next_compound_idx) && length(

71 tire_models) > 1 && race_params.

72 must_use_different_compound

73 warning('Could not find a different

74 compound for mandatory change! Strategy may be

75 invalid.');

76 next_compound_idx = start_compound_idx;

77 % Fallback, may violate rules

78 elseif isempty(next_compound_idx)

79 next_compound_idx = start_compound_idx;

80 % Only one compound defined

81 end

82 %Monte Carlo Simulation for (K, M)

83 num_sims = race_params.

84 num_simulations_for_expected_time;

85 simulation_times = zeros(num_sims, 1);

86 for i = 1:num_sims

87 % Initialize simulation variables for

88 this run

89 total_time_run = 0;

90 current_lap_run = 0;

91 laps_on_current_tire_run = 0;

92 current_compound_idx_run =

93 start_compound_idx;

94 pitted_run = false;

95 % --- Race Simulation Loop ---

96 while current_lap_run < race_params.

97 num_laps

98 current_lap_run = current_lap_run +

99 1;

100 laps_on_current_tire_run =

101 laps_on_current_tire_run + 1;

102 % Get tire model and calculate base

103 lap time + degradation

104 tire = tire_models(

105 current_compound_idx_run);

106 base_time = tire.base_lap_time;

107 % Ensure degradation function

108 handles potential non-vectorized inputs if

109 needed

110 try

111 degradation = tire.

112 degradation_func(laps_on_current_tire_run);

113 catch ME_deg

114 error('Error evaluating

115 degradation function for tire %s at age %d: %s',

116 tire.name, laps_on_current_tire_run, ME_deg.

117 message);

118 end

119 lap_time = base_time + degradation;

```

96         % Check for pit decision *for the
end of this lap*
97         pit_decision_this_lap = false;
98         pit_cost_this_lap = 0; % Cost
incurred *after* this lap
99
100         if ~pitted_run
101             % Check for SC event *this lap*
102             sc_occurred_this_lap = (rand() <
race_params.safety_car_probability_per_lap);
103
104             % Check for SC pit opportunity (
SC occurs AND we are at or past lap K)
105             if sc_occurred_this_lap &&
current_lap_run >= K
106                 pit_decision_this_lap = true
;
107                 pit_cost_this_lap =
race_params.pit_stop_time_loss_sc; % Free/cheap
pit
108                 % Check for mandatory pit (lap M
reached)
109                 elseif current_lap_run == M
110                     pit_decision_this_lap =
true;
111                     pit_cost_this_lap =
race_params.pit_stop_time_loss; % Normal cost
end
112
113                 % If pitting this lap, update
state for *next* lap
114                 if pit_decision_this_lap
115                     pitted_run = true;
116                     current_compound_idx_run =
next_compound_idx;
117                     laps_on_current_tire_run =
0; % Reset tire age for next lap
118                     % Pit cost is added *after*
the current lap time
end
119                     end
120
121                 % Add lap time and any pit cost from
the *previous* lap's decision
122                 total_time_run = total_time_run +
lap_time;
123
124                 if pit_decision_this_lap % Add cost
if pit decision was made for end of this lap
125                     total_time_run = total_time_run
+ pit_cost_this_lap;
126                 end
127
128             end % End while loop
129             simulation_times(i) = total_time_run;
130             end % End Monte Carlo loop
131
132             expected_time = mean(simulation_times);
133
134         else
135             error('Unknown strategy_type: %s',
strategy_type);
136         end
137     end
138
139     % simulate_race_fixed
140
141     function total_time = simulate_race_fixed(strategy,
race_params, tire_models)
142         % Simplified simulation for a fixed strategy
matrix (deterministic)

```

```

148     total_time = 0;
149     current_lap = 0;
150     num_stints = size(strategy, 1);
151
152     for stint_idx = 1:num_stints
153         compound_idx = strategy(stint_idx, 1);
154         stint_laps = strategy(stint_idx, 2);
155         tire = tire_models(compound_idx);
156
157         for lap_in_stint = 1:stint_laps
158             current_lap = current_lap + 1;
159             if current_lap > race_params.num_laps
160                 break;
161             end
162             tire_age = lap_in_stint;
163             lap_time = tire.base_lap_time + tire.
degradation_func(tire_age);
164             total_time = total_time + lap_time;
165         end
166
167         if stint_idx < num_stints && current_lap <=
race_params.num_laps
168             total_time = total_time + race_params.
pit_stop_time_loss;
169         end
170         if current_lap > race_params.num_laps, break
; end
171     end
172
173     if current_lap < race_params.num_laps
174         warning('Fixed strategy did not complete the
full race distance!');
175         total_time = Inf;
176     end
177 end

```

Listing 3: Function evaluating expected race time via Monte Carlo simulation.

D. fit_tire_model_from_csv.m

```

1 % --- File: define_data_driven_tire_models.m (New
File) ---
2 function tire_models =
define_data_driven_tire_models(csv_filepath,
target_driver, target_compound_name,
target_stint)
3 % Defines tire models by fitting data from a CSV
file for one compound,
4 % and uses placeholder models for others.
5
6 fprintf('--- Defining Data-Driven Tire Models
---\n');
7 tire_models = define_placeholder_tire_models();
8 % Start with placeholders
9
10 try
11     % Fit the target tire model
12     fitted_params = fit_tire_model_from_csv(
csv_filepath, target_driver,
target_compound_name, target_stint);
13
14     % Find the index corresponding to the target
compound name
15     compound_idx = -1;
16     for i = 1:length(tire_models)
17         % Use strcmpi for case-insensitive
comparison, check if name contains target
18         if contains(tire_models(i).name,
target_compound_name, 'IgnoreCase', true)
19             compound_idx = i;
20             break;
21         end
22     end

```



```

21     end
22
23     if compound_idx > 0
24         fprintf('Updating model for %s (Index %d
25 ) with data-driven parameters.\n', tire_models(
26 compound_idx).name, compound_idx);
27         tire_models(compound_idx).base_lap_time
28 = fitted_params.base_lap_time;
29         tire_models(compound_idx).
30 degradation_func = fitted_params.
31 degradation_func;
32         tire_models(compound_idx).data_driven =
33 true; % Add a flag
34     else
35         warning('Could not find placeholder
36 model matching target compound name "%s". Using
37 only placeholders.', target_compound_name);
38     end
39
40 catch ME
41     warning('Error fitting data-driven model for
42 %s: %s. Using placeholder models.',
43 target_compound_name, ME.message);
44     % Keep using the placeholder models defined
45 initially
46 end
47
48 disp('Tire Models Defined.');
```

```

37 for i = 1:length(tire_models)
38     is_data_driven = isfield(tire_models(i), '
39 data_driven') && tire_models(i).data_driven;
40     fprintf(' Model %d: %s (Data-Driven: %s)\n'
41 , i, tire_models(i).name, mat2str(is_data_driven
42 ));
43 end
44 fprintf('
45 -----\n');
```

```

46 end

```

Listing 4: Function for defining tire models.